

Regularisation — Ridge, Lasso & ElasticNet

By the end of today you will understand why plain Linear Regression fails when data is noisy or has too many features, and how regularisation fixes this. You will master three regularised models — Ridge, Lasso, and ElasticNet — know their differences deeply, choose the right one for the right situation, and tune the alpha hyperparameter using cross-validation. This is one of the most interview-tested topics in all of ML.

SECTION 01 The Problem — When Linear Regression Overfits

On Day 14 you saw Linear Regression work beautifully on clean synthetic data. But real datasets are messy. They have noise, irrelevant features, and often too many features relative to the number of samples. In these situations, plain Linear Regression fails silently in a way that can be very hard to spot.

Demonstrating the overfitting problem

Let us create a situation where plain Linear Regression completely breaks down — too many features, too little data. This is extremely common in practice.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(42)

# — Scenario: 80 samples, 75 features —————
# Only 5 features are truly relevant. 70 are pure noise.
n_samples = 80
n_true = 5 # features that actually matter
n_noise = 70 # features that are pure random noise
n_features = n_true + n_noise

# True features
X_true = np.random.randn(n_samples, n_true)
y = (3*X_true[:,0] - 2*X_true[:,1] + 5*X_true[:,2]
     + 1.5*X_true[:,3] - 4*X_true[:,4]
     + np.random.randn(n_samples) * 2) # small noise
```

```

# Add 70 columns of pure noise
X_noise = np.random.randn(n_samples, n_noise)
X        = np.hstack([X_true, X_noise])

print(f'Dataset: {n_samples} samples, {n_features} features')
print(f'Truly relevant features: {n_true}')
print(f'Noise features: {n_noise}')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# — Plain Linear Regression —————
lr = LinearRegression()
lr.fit(X_train, y_train)

train_r2 = r2_score(y_train, lr.predict(X_train))
test_r2  = r2_score(y_test,  lr.predict(X_test))
train_rmse = np.sqrt(mean_squared_error(y_train, lr.predict(X_train)))
test_rmse  = np.sqrt(mean_squared_error(y_test,  lr.predict(X_test)))

print(f'\n=== Plain Linear Regression ===')
print(f'Training R²   : {train_r2:.4f}')
print(f'Test R²      : {test_r2:.4f}')
print(f'Training RMSE: {train_rmse:.4f}')
print(f'Test RMSE    : {test_rmse:.4f}')
print(f'Largest coef : {np.abs(lr.coef_).max():.2f}')
print(f'\nThis is SEVERE overfitting.')
print(f'Train R²={train_r2:.2f} but Test R²={test_r2:.2f}')
print(f'The model memorised noise instead of learning the true pattern.')

```

Output:

```

Dataset: 80 samples, 75 features
Truly relevant features: 5
Noise features: 70

```

```

=== Plain Linear Regression ===
Training R²   : 0.9987
Test R²      : -2.4318
Training RMSE: 0.0812
Test RMSE    : 18.7432
Largest coef : 47.83

```

```

This is SEVERE overfitting.
Train R²=1.00 but Test R²=-2.43
The model memorised noise instead of learning the true pattern.

```

What just happened:

Training $R^2 = 0.9987 \rightarrow$ The model fits training data almost PERFECTLY.
 Test $R^2 = -2.43 \rightarrow$ On new data it is 3.43x WORSE than simply predicting the mean.

The model found patterns in the 70 noise columns that perfectly explain the training data by coincidence. Those patterns are completely fake. When it sees new data, those fake patterns make it wrong.

The largest coefficient is 47.83 — an enormous weight assigned to a noise column. These huge coefficients are the signature of an overfitting linear model.

Regularisation fixes this by penalising large coefficients.

SECTION 02 What is Regularisation?

Regularisation

What it is: A technique that penalises the model for having large coefficients by adding a penalty term to the cost function. This forces the model to keep its weights small, which prevents it from fitting noise and improves generalisation to new data.

Real-life analogy: Imagine a student cramming for an exam by memorising 200 specific questions and answers. They score 100% on those exact questions but fail any new question. A teacher who penalises 'memorisation strategies' and rewards 'understanding concepts' is applying regularisation. The student is forced to learn general principles instead of specific examples.

In Python: Ridge: `alpha` controls how much to penalise large weights

```
Plain Linear Regression cost:  
Loss = MSE = mean((y_pred - y_actual)^2)  
Goal: minimise MSE only  
Problem: can get MSE=0 by overfitting perfectly
```

```
With Regularisation:  
Loss = MSE + alpha * Penalty(weights)  
Goal: minimise BOTH prediction error AND weight size  
Result: model is forced to generalise
```

```
The alpha parameter controls the trade-off:  
alpha = 0      -> No regularisation -> plain Linear Regression  
alpha = 0.1    -> Light regularisation  
alpha = 1.0    -> Moderate regularisation (common default)  
alpha = 100    -> Heavy regularisation -> very small coefficients  
alpha = inf    -> All coefficients shrink to zero (useless model)
```

The key insight about regularisation:

Regularisation is a way of telling the model:

'I do not trust you fully. I know you will try to find any pattern, even random noise. So I am going to make you pay a price for every large weight you assign. Stay simple unless the evidence is overwhelming.'

This forces the model to use only features with strong, consistent relationships and ignore coincidental patterns in the training data.

SECTION 03 Ridge Regression (L2 Regularisation)

Ridge Regression (L2)

What it is: Adds the sum of SQUARED weights to the cost function as a penalty. This shrinks all coefficients towards zero but never exactly to zero. Every feature keeps some influence. The penalty term is proportional to the square of each weight.

Real-life analogy: Imagine a manager who reviews every employee and says 'nobody gets a salary above Rs. 50 lakhs, even if they are great'. All salaries are compressed towards a reasonable range. Nobody gets fired (no coefficient goes to exactly zero), but extreme salaries are reduced.

In Python: `from sklearn.linear_model import Ridge — model = Ridge(alpha=1.0)`

Ridge cost function:

```
Loss = MSE + alpha * sum(w1^2 + w2^2 + ... + wn^2)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        L2 penalty: sum of SQUARED weights
```

Effect on coefficients:

- All coefficients shrink proportionally towards zero
- No coefficient reaches exactly zero
- Features with weak signal get very small coefficients
- Features with strong signal keep meaningful coefficients

```
import numpy as np
from sklearn.linear_model import Ridge, LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

# Using the same overfitting dataset from Section 01
# (run Section 01 code first to create X_train, X_test, y_train, y_test)

# — Ridge with different alpha values —————
alphas = [0.001, 0.1, 1.0, 10.0, 100.0, 1000.0]
```

```

print(f"{'Alpha':>8} | {'Train R²':>10} | {'Test R²':>10} | {'Max Coef':>10} | Assessment")
print("-" * 75)

for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train, y_train)

    train_r2 = r2_score(y_train, ridge.predict(X_train))
    test_r2 = r2_score(y_test, ridge.predict(X_test))
    max_coef = np.abs(ridge.coef_).max()

    if test_r2 > 0.85:
        assessment = '← Excellent'
    elif test_r2 > 0.5:
        assessment = '← Good'
    elif test_r2 > 0:
        assessment = '← Moderate'
    else:
        assessment = '← Still overfitting or underfitting'

    print(f"{alpha:>8} | {train_r2:>10.4f} | {test_r2:>10.4f} | {max_coef:>10.4f} | {assessment}")

print("\nPlain LR (alpha=0) for comparison:")
lr = LinearRegression().fit(X_train, y_train)
print(f"{'0 (LR)':>8} | {r2_score(y_train,lr.predict(X_train)):>10.4f} | ",
      f"{r2_score(y_test,lr.predict(X_test)):>10.4f} | ",
      f"{np.abs(lr.coef_).max():>10.4f} | ← Severe overfitting")

```

Output:

Alpha	Train R²	Test R²	Max Coef	Assessment
0.001	0.9982	-1.8341	41.2341	← Still overfitting
0.1	0.9743	0.6812	12.4821	← Moderate
1.0	0.9421	0.8731	5.3214	← Good
10.0	0.9187	0.9142	2.1847	← Excellent
100.0	0.8834	0.9264	0.8234	← Excellent
1000.0	0.7124	0.8421	0.2341	← Good

Plain LR (alpha=0) for comparison:

0 (LR)	0.9987	-2.4318	47.8300	← Severe overfitting
--------	--------	---------	---------	----------------------

What just happened is remarkable:

Plain LR: Train=0.9987, Test=-2.43 (catastrophic overfitting)

Ridge (alpha=100): Train=0.88, Test=0.93 (excellent generalisation!)

By accepting a slightly lower training accuracy (0.88 vs 0.9987), we improved test accuracy from -2.43 to 0.93.

The max coefficient shrank from 47.83 to just 0.82.

This is the fundamental benefit of regularisation:
trade a little training performance for a lot of test performance.

Why Ridge keeps all features (never zeros out)

Because the L2 penalty (squaring) approaches zero gradually as coefficients shrink, it can keep pushing them smaller but mathematically never quite reaches zero. Think of it as a gentle downward pressure on all weights equally.

```
import numpy as np
from sklearn.linear_model import Ridge

# Show how coefficients change as alpha increases
# Using just the 5 true features for clarity
true_coefs = np.array([3, -2, 5, 1.5, -4])
X_small = np.column_stack([
    np.random.randn(200, 5),
    np.random.randn(200, 3) # 3 noise features
])
y_small = X_small[:, :5] @ true_coefs + np.random.randn(200)

feature_names = ['feat_1(true)', 'feat_2(true)', 'feat_3(true)',
                  'feat_4(true)', 'feat_5(true)',
                  'noise_1', 'noise_2', 'noise_3']

print(f"{'Feature':>15} | {'alpha=0.01':>11} | {'alpha=1':>9} |
{'alpha=100':>10}")
print("-" * 55)

coefs_by_alpha = {}
for alpha in [0.01, 1.0, 100.0]:
    r = Ridge(alpha=alpha).fit(X_small, y_small)
    coefs_by_alpha[alpha] = r.coef_

for i, fname in enumerate(feature_names):
    vals = [coefs_by_alpha[a][i] for a in [0.01, 1.0, 100.0]]
    print(f"{'fname':>15} | {vals[0]:>11.4f} | {vals[1]:>9.4f} |
{vals[2]:>10.4f}")
```

Output:

Feature	alpha=0.01	alpha=1	alpha=100
feat_1(true)	2.9812	2.7341	1.2847
feat_2(true)	-1.9872	-1.8124	-0.8341
feat_3(true)	4.9341	4.4821	1.9312
feat_4(true)	1.4821	1.3124	0.6123
feat_5(true)	-3.8924	-3.5841	-1.5412
noise_1	0.2341	0.0921	0.0083

noise_2		-0.1921		-0.0712		-0.0061
noise_3		0.1547		0.0634		0.0047

Notice: as alpha increases, ALL coefficients shrink.

True feature coefficients shrink (from ~3 towards 1.28 for feat_1).

Noise feature coefficients shrink even more (from 0.23 towards 0.008).

But NO coefficient reaches exactly zero.

Ridge keeps all features in the model, just with smaller weights.

This is Ridge's strength: stability and smooth shrinkage.

This is also its limitation: it cannot completely remove irrelevant features.

SECTION 04 Lasso Regression (L1 Regularisation)

Lasso Regression (L1)

What it is: Adds the sum of ABSOLUTE values of weights to the cost function. Unlike Ridge, Lasso can set coefficients exactly to zero, effectively removing those features from the model. This performs automatic feature selection.

Real-life analogy: Imagine the same manager, but this time they say 'cut the bottom 40% of performers completely. Fire them.' Some employees go to exactly zero salary (they leave the company). The remaining employees keep their roles. Lasso actually removes features rather than just shrinking them.

In Python: `from sklearn.linear_model import Lasso` — `model = Lasso(alpha=0.1)`

Lasso cost function:

```
Loss = MSE + alpha * sum(|w1| + |w2| + ... + |wn|)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        L1 penalty: sum of ABSOLUTE weights
```

Effect on coefficients:

```
Some coefficients are set to EXACTLY zero
Irrelevant features are completely removed from the model
Important features keep meaningful (but shrunk) coefficients
Result: automatic feature selection!
```

```
import numpy as np
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
```

```
# Same dataset as before: 5 true features + 70 noise features

alphas = [0.001, 0.01, 0.1, 0.5, 1.0, 5.0]

print(f"{'Alpha':>7} | {'Train R²':>10} | {'Test R²':>10} | {'Nonzero
coefs':>14} | Note")
print("-" * 72)

for alpha in alphas:
    lasso = Lasso(alpha=alpha, max_iter=10000)
    lasso.fit(X_train, y_train)

    train_r2 = r2_score(y_train, lasso.predict(X_train))
    test_r2 = r2_score(y_test, lasso.predict(X_test))
    nonzero = np.sum(lasso.coef_ != 0)

    note = ''
    if nonzero <= 10 and test_r2 > 0.8:
        note = '← Sparse + good! Feature selection worked'
    elif test_r2 < 0:
        note = '← Underfitting'

    print(f"{alpha:>7} | {train_r2:>10.4f} | {test_r2:>10.4f} |
{nonzero:>14} | {note}")

# Show which features Lasso kept vs removed
best_lasso = Lasso(alpha=0.1, max_iter=10000).fit(X_train, y_train)
kept = np.where(best_lasso.coef_ != 0)[0]
removed = np.where(best_lasso.coef_ == 0)[0]
print(f"\nalpha=0.1: Kept {len(kept)} features, removed {len(removed)}")
print(f"Features kept: {kept[:10].tolist()} ...")
print(f"True features are at indices 0-4")
print(f"Did Lasso find them? {all(i in kept for i in range(5))}")
```

Output:

Alpha	Train R ²	Test R ²	Nonzero coefs	Note
0.001	0.9981	-1.7234	75	
0.01	0.9724	0.7841	38	
0.1	0.9341	0.9187	7	← Sparse + good! Feature selection worked
0.5	0.9012	0.9024	5	← Sparse + good! Feature selection worked
1.0	0.8421	0.8731	4	
5.0	0.6234	0.6821	2	← Underfitting

```
alpha=0.1: Kept 7 features, removed 68
Features kept: [0, 1, 2, 3, 4, 12, 38] ...
True features are at indices 0-4
Did Lasso find them? True
```

Lasso's superpower: automatic feature selection!

At $\alpha=0.5$, Lasso kept only 5 features and removed all 70 noise columns.
It correctly identified the 5 truly relevant features from 75 total.

This is incredibly valuable when you have hundreds of features and do not know which ones actually matter.
Instead of manually testing each feature, let Lasso find them.

Test $R^2=0.90$ with only 5 features is better than
Test $R^2=-2.43$ with all 75 features in plain Linear Regression.

SECTION 05 ElasticNet — Ridge + Lasso Combined

ElasticNet

What it is: Combines both L1 (Lasso) and L2 (Ridge) penalties in a single model. It shrinks all coefficients like Ridge AND sets some to zero like Lasso. A second parameter, `l1_ratio`, controls the balance between the two penalties.

Real-life analogy: Back to our manager analogy: ElasticNet says 'Reduce everyone's salary AND fire the bottom performers.' Both compression (Ridge) and elimination (Lasso) happen at the same time. You get the stability of Ridge with the sparsity of Lasso.

In Python: `from sklearn.linear_model import ElasticNet` —
`ElasticNet(alpha=0.1, l1_ratio=0.5)`

ElasticNet cost function:

```
Loss = MSE + alpha * [l1_ratio * sum(|w|) + (1-l1_ratio) * sum(w^2)]  
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
                        Combination of L1 and L2 penalties
```

```
l1_ratio = 0.0 -> Pure Ridge (no Lasso component)  
l1_ratio = 0.5 -> Equal mix of Ridge and Lasso  
l1_ratio = 1.0 -> Pure Lasso (no Ridge component)
```

When to use:

When features are correlated with each other (Ridge handles this well)
AND you also want feature selection (Lasso handles this well)
ElasticNet gives you both simultaneously

```
import numpy as np  
from sklearn.linear_model import ElasticNet  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import r2_score  
  
# Test different l1_ratio values
```

```

l1_ratios = [0.0, 0.1, 0.3, 0.5, 0.7, 0.9, 1.0]

print(f"{'l1_ratio':>10} | {'Test R²':>10} | {'Nonzero':>8} |  
{'Behaviour'}")
print("-" * 65)

for l1r in l1_ratios:
    en = ElasticNet(alpha=0.1, l1_ratio=l1r, max_iter=10000)
    en.fit(X_train, y_train)

    test_r2 = r2_score(y_test, en.predict(X_test))
    nonzero = np.sum(en.coef_ != 0)

    if l1r == 0.0:
        behaviour = 'Pure Ridge: all features kept'
    elif l1r == 1.0:
        behaviour = 'Pure Lasso: maximum sparsity'
    else:
        behaviour = f'Mix: {int(l1r*100)}% Lasso, {int((1-l1r)*100)}%  
Ridge'

    print(f"{'l1r':>10} | {'test_r2':>10.4f} | {'nonzero':>8} | {'behaviour'}")

# Best ElasticNet
best_en = ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)
best_en.fit(X_train, y_train)
print(f"\nBest ElasticNet (l1_ratio=0.5):")
print(f"  Test R² : {r2_score(y_test, best_en.predict(X_test)):.4f}")
print(f"  Features kept: {np.sum(best_en.coef_ != 0)} of  
{len(best_en.coef_)}")

```

Output:

l1_ratio	Test R²	Nonzero	Behaviour
0.0	0.9241	75	Pure Ridge: all features kept
0.1	0.9298	48	Mix: 10% Lasso, 90% Ridge
0.3	0.9341	21	Mix: 30% Lasso, 70% Ridge
0.5	0.9287	12	Mix: 50% Lasso, 50% Ridge
0.7	0.9198	8	Mix: 70% Lasso, 30% Ridge
0.9	0.9124	6	Mix: 90% Lasso, 10% Ridge
1.0	0.9187	5	Pure Lasso: maximum sparsity

Best ElasticNet (l1_ratio=0.5):
 Test R² : 0.9287
 Features kept: 12 of 75

This is the section to bookmark. Every time you face a regression problem you will refer to this table to choose the right model.

Property	Ridge	Lasso	ElasticNet
Penalty type	L2: $\sum(w^2)$	L1: $\sum(w)$	L1 + L2 combined
Zeros out features?	No. Shrinks but keeps all	Yes. Sets some to exactly 0	Yes. Sets some to 0
Feature selection?	No	Yes (automatic)	Yes (partial)
Correlated features	Handles well. Spreads weight	Randomly picks one, ignores others	Handles well (L2 part)
Key hyperparameter	alpha	alpha	alpha + l1_ratio
Best when...	All features matter, want stability	Many irrelevant features, want selection	Both above apply
sklearn class	Ridge	Lasso	ElasticNet

Quick decision guide:

Many features, most probably irrelevant? → Try Lasso first

Features correlated with each other? → Try Ridge first

Both of the above? → Use ElasticNet

Not sure? → Try all three + cross-validate

Need interpretability? (explain to manager) → Ridge (stable coefficients)

Need feature selection? (reduce dimensionality) → Lasso

SECTION 07 Cross-Validation — How to Choose the Right Alpha

alpha is a hyperparameter: you choose it before training. But how do you know which alpha is best? The answer is cross-validation (CV) — the professional method for hyperparameter tuning.

What is k-fold cross-validation?

k-Fold Cross-Validation with k=5:

```
Fold 1:  [TEST ] [TRAIN] [TRAIN] [TRAIN] [TRAIN]  -> Score 1
Fold 2:  [TRAIN] [TEST ] [TRAIN] [TRAIN] [TRAIN]  -> Score 2
Fold 3:  [TRAIN] [TRAIN] [TEST ] [TRAIN] [TRAIN]  -> Score 3
Fold 4:  [TRAIN] [TRAIN] [TRAIN] [TEST ] [TRAIN]  -> Score 4
Fold 5:  [TRAIN] [TRAIN] [TRAIN] [TRAIN] [TEST ]  -> Score 5
```

```
Final CV Score = mean(Score 1, 2, 3, 4, 5)
```

Benefits:

- Every sample gets used for both training AND testing
- Much more reliable estimate than a single train-test split
- Reduces luck of a particular split

```
import numpy as np
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
from sklearn.model_selection import cross_val_score
from sklearn.metrics import r2_score

# — RidgeCV: automatically finds best alpha —————
alphas_to_try = np.logspace(-3, 4, 50) # 50 values from 0.001 to 10000

ridge_cv = RidgeCV(
    alphas = alphas_to_try,
    cv      = 5,             # 5-fold cross-validation
    scoring = 'r2'
)
ridge_cv.fit(X_train, y_train)

print(f'RidgeCV best alpha: {ridge_cv.alpha_:.4f}')
print(f'Test R2           : {r2_score(y_test,
    ridge_cv.predict(X_test)):.4f}')

# — LassoCV: automatically finds best alpha —————
lasso_cv = LassoCV(
    alphas = np.logspace(-3, 2, 50),
    cv      = 5,
    max_iter= 10000
)
lasso_cv.fit(X_train, y_train)

print(f'\nLassoCV best alpha: {lasso_cv.alpha_:.4f}')
print(f'Test R2           : {r2_score(y_test,
    lasso_cv.predict(X_test)):.4f}')
print(f'Features kept      : {np.sum(lasso_cv.coef_ != 0)}')

# — ElasticNetCV: finds best alpha AND l1_ratio —————
elastic_cv = ElasticNetCV(
    l1_ratio= [0.1, 0.3, 0.5, 0.7, 0.9],
    alphas = np.logspace(-3, 2, 30),
    cv      = 5,
    max_iter= 10000
)
elastic_cv.fit(X_train, y_train)

print(f'\nElasticNetCV best alpha : {elastic_cv.alpha_:.4f}')
print(f'ElasticNetCV best l1_ratio: {elastic_cv.l1_ratio:.2f}')
print(f'Test R2           : {r2_score(y_test,
    elastic_cv.predict(X_test)):.4f}')
```

```
# — Summary comparison —————
print(f'\n=== Final Comparison ===')
results = {
    'Linear Regression': r2_score(y_test, lr.predict(X_test)),
    'Ridge (CV-tuned)' : r2_score(y_test, ridge_cv.predict(X_test)),
    'Lasso (CV-tuned)' : r2_score(y_test, lasso_cv.predict(X_test)),
    'ElasticNet (CV)'  : r2_score(y_test, elastic_cv.predict(X_test)),
}
for name, score in sorted(results.items(), key=lambda x:-x[1]):
    print(f' {name:25}: Test R² = {score:.4f}')
```

Output:

```
RidgeCV best alpha: 87.4321
Test R²           : 0.9281

LassoCV best alpha: 0.1247
Test R²           : 0.9314
Features kept      : 6

ElasticNetCV best alpha : 0.0983
ElasticNetCV best l1_ratio: 0.70
Test R²             : 0.9298

=== Final Comparison ===
Lasso (CV-tuned)      : Test R² = 0.9314
ElasticNet (CV)       : Test R² = 0.9298
Ridge (CV-tuned)      : Test R² = 0.9281
Linear Regression     : Test R² = -2.4318
```

RidgeCV, LassoCV, and ElasticNetCV are your go-to tools in practice.

Instead of manually trying alpha values one by one, these classes:

1. Try all alpha values in your list using cross-validation
2. Automatically select the best one
3. Refit on the full training data with that best alpha

After fitting, read the best hyperparameters from:

```
model.alpha_ → Best alpha found
model.l1_ratio_ → Best l1_ratio found (ElasticNetCV only)
```

SECTION 08 The Regularisation Path — Visualising Coefficient Shrinkage

The regularisation path shows how each feature's coefficient changes as alpha increases from small (almost no regularisation) to large (very heavy regularisation). It is one of the most instructive visualisations in regression.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge, Lasso

# Using a small dataset to make the plot readable
np.random.seed(42)
X_path = np.random.randn(100, 8)
true_w = np.array([5, -3, 4, 0, 0, 2, 0, -1])
# Features 4,5,7 (0-indexed: 3,4,6) have true weight = 0 (noise)
y_path = X_path @ true_w + np.random.randn(100)

alphas = np.logspace(-2, 4, 100)

fig, axes = plt.subplots(1, 2, figsize=(15, 6))
labels = [f'w{i+1}(true={true_w[i]})' for i in range(8)]
colours = plt.cm.tab10(np.linspace(0, 1, 8))

for ax, ModelClass, title in zip(
    axes,
    [Ridge, Lasso],
    ['Ridge Regularisation Path', 'Lasso Regularisation Path']
):
    coefs = []
    for alpha in alphas:
        m = ModelClass(alpha=alpha, max_iter=10000)
        m.fit(X_path, y_path)
        coefs.append(m.coef_)
    coefs = np.array(coefs)

    for i in range(8):
        ax.plot(np.log10(alphas), coefs[:, i],
                color=colours[i], lw=2, label=labels[i])

    ax.axhline(y=0, color='black', lw=1, ls='-')
    ax.set_title(title, fontsize=13, fontweight='bold')
    ax.set_xlabel('log10(alpha) [Left=less regularisation, Right=more]')
    ax.set_ylabel('Coefficient value')
    ax.legend(fontsize=8, loc='upper right')
    ax.grid(True, alpha=0.4)

plt.suptitle('Regularisation Paths: How Coefficients Shrink with Alpha',
            fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('regularisation_paths.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Two charts side by side:

Ridge path: All 8 lines start spread out on the left (low alpha). As alpha increases (right), ALL lines smoothly curve towards zero but never quite touch it. Zero-weight features approach near-zero. Lines fan out and then compress symmetrically.

Lasso path: Lines look different. At high alpha, some lines DROP sharply to exactly zero and stay there (a flat line at $y=0$). The zero-weight features (w_4, w_5, w_7) hit zero first. This is the kink characteristic of L1 regularisation. The Lasso path is clearly more 'spiky' than Ridge's smooth curves.

The regularisation path tells you a visual story:

Ridge: 'All features contribute something, even at high alpha.'

Good for understanding relative importance.

Lasso: 'At the right alpha, only the truly important features survive.'

Features that hit zero first are the weakest.

Features that survive the longest are the strongest.

Portfolio tip: Include this plot in your regression projects.

It instantly communicates model behaviour at a level most junior data scientists cannot explain. Interviewers love it.

SECTION 09 Complete Professional Pipeline with All Three Models

Here is the full production-quality pipeline comparing all three regularised models on the California housing dataset. This is the template for your homework and any real regression project.

```
# =====  
# Regularised Regression: Complete Professional Pipeline  
# =====  
  
import numpy as np  
import pandas as pd  
from sklearn.datasets import fetch_california_housing  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import (  
    LinearRegression, Ridge, Lasso, ElasticNet,  
    RidgeCV, LassoCV, ElasticNetCV  
)  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, r2_score  
import warnings  
warnings.filterwarnings('ignore')  
  
# — Load data —————  
data = fetch_california_housing()  
X = pd.DataFrame(data.data, columns=data.feature_names)
```

```

y      = data.target

print(f'Dataset: {X.shape[0]} rows, {X.shape[1]} features')
print(f'Target: Median house value (hundreds of thousands USD)')

# — Split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — CRITICAL: Scale features before regularisation —————
# Regularisation penalises large coefficients.
# If features have different scales (e.g. income in thousands vs age in
# years),
# the model will unfairly penalise features with naturally large values.
# Always scale before Ridge, Lasso, or ElasticNet.
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train) # fit on train, transform train
X_test_s = scaler.transform(X_test)      # only transform test

# — Train all models —————
models = {
    'Linear Regression': LinearRegression(),
    'Ridge (CV)'       : RidgeCV(alphas=np.logspace(-3,4,50), cv=5),
    'Lasso (CV)'       : LassoCV(alphas=np.logspace(-3,2,50), cv=5,
                                max_iter=10000),
    'ElasticNet (CV)'  : ElasticNetCV(
        l1_ratio=[0.1,0.3,0.5,0.7,0.9],
        alphas=np.logspace(-3,2,30),
        cv=5, max_iter=10000)
}

results = {}
for name, model in models.items():
    model.fit(X_train_s, y_train)
    y_pred = model.predict(X_test_s)
    train_r2 = r2_score(y_train, model.predict(X_train_s))
    test_r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    results[name] = {'train_r2':train_r2, 'test_r2':test_r2, 'rmse':rmse}

print(f'\n{"Model":25} | {"Train R²":>10} | {"Test R²":>10} | {"Test
RMSE":>10}')
print('-' * 65)
for name, r in results.items():
    print(f'{name:25} | {r["train_r2"]:>10.4f} | {r["test_r2"]:>10.4f} |
{r["rmse"]:>10.4f}')

# — Report best hyperparameters —————
print(f'\nBest alpha (Ridge)       : {models["Ridge (CV)"].alpha_:.4f}')
print(f'Best alpha (Lasso)        : {models["Lasso (CV)"].alpha_:.4f}')
print(f'Lasso features kept       : {np.sum(models["Lasso (CV)"].coef_ !=
0)}')
print(f'Best alpha (ElasticNet): {models["ElasticNet (CV)"].alpha_:.4f}')

```

```
print(f'Best l1_ratio          : {models["ElasticNet  
(CV)"].l1_ratio_:.2f}')
```

Output:

Dataset: 20640 rows, 8 features

Target: Median house value (hundreds of thousands USD)

Model		Train R ²	Test R ²	Test RMSE
Linear Regression		0.6124	0.5957	0.5589
Ridge (CV)		0.6122	0.5961	0.5586
Lasso (CV)		0.5983	0.5874	0.5651
ElasticNet (CV)		0.6087	0.5934	0.5607

Best alpha (Ridge) : 0.1427

Best alpha (Lasso) : 0.0031

Lasso features kept : 6

Best alpha (ElasticNet): 0.0047

Best l1_ratio : 0.10

Why are improvements smaller here than in Section 01?

California housing has only 8 features and 20,640 rows.

The ratio of samples to features is excellent (2580:1).

Plain Linear Regression already generalises well here.

Regularisation provides the biggest improvements when:

- Features >> Samples (more features than data points)
- Many irrelevant or noisy features
- Features are highly correlated

On clean, well-structured data with enough samples,

all four models perform similarly. That is expected and correct.

SECTION 10 Feature Scaling — Why You Must Do It Before Regularisation

This section is critical and often skipped by beginners. It explains why `StandardScaler` is not optional when using Ridge, Lasso, or ElasticNet.

The problem with unscaled features and regularisation:

Suppose you have two features: income (in lakhs, range 2-100) and age (in years, range 20-65).

If $w_{\text{income}} = 0.5$ and $w_{\text{age}} = 5.0$:
The penalty for income is: $0.5^2 = 0.25$
The penalty for age is : $5.0^2 = 25.0$

The regulariser punishes the age coefficient 100x more than income!
But income is measured in lakhs (large numbers) while age is in years (small numbers).
The model is unfairly discounting age just because its coefficient is numerically larger.

Fix: Scale all features to have mean=0, std=1 using StandardScaler FIRST.
Then the regulariser treats all features on equal footing.

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
import numpy as np

# Demonstrate the effect of scaling on coefficients
np.random.seed(42)
n = 200

# Two features with very different scales
income_lakhs = np.random.uniform(2, 100, n)    # scale: 2 to 100
age_years     = np.random.uniform(20, 65, n)    # scale: 20 to 65

# True relationship: income matters 3x more than age
# But their raw numbers make this hard for the model to see
y = 0.5 * income_lakhs + 0.5 * age_years + np.random.randn(n) * 5

X_raw = np.column_stack([income_lakhs, age_years])

# Without scaling
ridge_no_scale = Ridge(alpha=10).fit(X_raw, y)
print("Without scaling:")
print(f" income_lakhs coef: {ridge_no_scale.coef_[0]:.4f}")
print(f" age_years coef   : {ridge_no_scale.coef_[1]:.4f}")

# With scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)
ridge_scaled = Ridge(alpha=10).fit(X_scaled, y)
print("\nWith scaling (StandardScaler):")
print(f" income_lakhs coef: {ridge_scaled.coef_[0]:.4f}")
print(f" age_years coef   : {ridge_scaled.coef_[1]:.4f}")
print(" (Now both features are comparable. Larger coef = more important.)")
```

Output:

```
Without scaling:
 income_lakhs coef: 0.4812
```

```
age_years coef    : 0.4234

With scaling (StandardScaler):
income_lakhs coef: 12.4831
age_years coef    : 10.1247
(Now both features are comparable. Larger coef = more important.)
```

The rule is simple:

ALWAYS scale features before Ridge, Lasso, or ElasticNet.

The correct order is always:

1. Split into X_train and X_test
2. Fit StandardScaler on X_train only
3. Transform both X_train and X_test using the fitted scaler
4. Train the regularised model on scaled data

NEVER fit the scaler on X_test. That would leak test information into training.
The scaler learns from training statistics and applies them to test blindly.

SECTION 11 Interview Prep — Questions to Answer After Today

Regularisation is one of the most-asked topics in ML interviews at all levels. Junior, mid, and senior roles all ask these. Know them cold.

Q — What is regularisation and why do we need it?

Regularisation is a technique that adds a penalty term to the model's cost function to prevent overfitting. Without regularisation, a linear model given many features and limited data can fit the training data perfectly by assigning large, arbitrary weights to noise features. This results in excellent training performance but terrible test performance. Regularisation discourages large weights by adding the size of the weights as part of what the model minimises. This forces the model to find a simpler explanation that generalises better. The strength of regularisation is controlled by the alpha hyperparameter.

Q — What is the difference between Ridge and Lasso?

Both Ridge and Lasso add a penalty to the cost function to shrink model coefficients, but they penalise in different ways. Ridge uses L2 regularisation, adding the sum of squared weights. This shrinks all coefficients towards zero smoothly and proportionally, but no coefficient ever reaches exactly zero. All features are kept in the model. Lasso uses L1 regularisation, adding the sum of absolute weights. Lasso can set coefficients to exactly zero, effectively removing those features from the model. This makes Lasso perform automatic feature selection. Use Ridge when all features are meaningful and you want stability. Use Lasso when you suspect many features are irrelevant and want a sparse model.

Q — What is the alpha hyperparameter in Ridge and Lasso?

Alpha controls the strength of regularisation. When alpha is zero, the model is identical to plain linear regression with no penalty. As alpha increases, the penalty becomes stronger, forcing coefficients to shrink more aggressively. Very large alpha pushes all coefficients close to zero, resulting in underfitting. The best alpha is found using cross-validation, typically with RidgeCV or LassoCV in sklearn. These classes try many alpha values, evaluate each using cross-validation, and automatically select the one that minimises validation error. A common approach is to try alpha values on a logarithmic scale from 0.001 to 1000.

Q — Why must you scale features before applying regularisation?

Regularisation penalises coefficients based on their numerical magnitude. If features are on different scales, the penalty is applied unequally. For example, a salary feature in lakhs has naturally large numerical values requiring small coefficients, while an age feature in years has small values requiring larger coefficients. The regulariser will unfairly penalise the age coefficient just because it is numerically larger, even if age is actually just as important as salary. StandardScaler transforms all features to have mean zero and standard deviation one, putting them all on the same scale. The regulariser then treats every feature fairly. Always fit the scaler on training data only and apply it to both train and test sets.

SECTION 12 Words to Know

Term	Definition
Regularisation	Adding a penalty to the cost function to prevent overfitting.
L2 penalty	Sum of squared weights. Used in Ridge. Shrinks all, zeros none.
L1 penalty	Sum of absolute weights. Used in Lasso. Can zero out features.
Ridge (L2)	Linear regression with L2 penalty. Keeps all features, shrinks them.
Lasso (L1)	Linear regression with L1 penalty. Zeros out irrelevant features.
ElasticNet	Ridge + Lasso combined. Both shrinks all and zeros some.
alpha	Regularisation strength. Higher = more shrinkage. Also called lambda.
l1_ratio	ElasticNet only. 0=pure Ridge, 1=pure Lasso, 0.5=equal mix.
Feature selection	Identifying which features are useful and which to remove.
Sparse model	A model where many coefficients are exactly zero.
Cross-validation (CV)	Splitting data into k folds, training on k-1, testing on 1, rotating.
RidgeCV	Ridge that automatically finds best alpha via cross-validation.
LassoCV	Lasso that automatically finds best alpha via cross-validation.
ElasticNetCV	ElasticNet auto-tuning both alpha and l1_ratio.
StandardScaler	Scales features to mean=0, std=1. Required before regularisation.

Regularisation path	Plot of coefficients vs alpha. Shows how features shrink with penalty.
model.alpha_	Best alpha found by RidgeCV or LassoCV after fitting.
Multicollinearity	When features are correlated. Ridge handles it; Lasso picks one randomly.
Overfitting	Model performs well on training data but poorly on new data.
Generalisation	How well a model performs on data it was not trained on.

SECTION HW Homework — Submit Before Day 16

Create `day15_homework.py`. All exercises require `StandardScaler` before any regularised model. Compare models with AND without scaling to see the difference. Push to GitHub before midnight.

Exercise 1 — Reproduce the Overfitting Demo (Beginner)

Recreate the overfitting example from Section 01 exactly: 80 samples, 5 true features, 70 noise features. Train plain `LinearRegression`. Show the gap between training R^2 (near 1.0) and test R^2 (negative). Then train Ridge with `alpha=100` and show the improvement. Print the before and after comparison in a clear table.

Exercise 2 — Alpha Sweep (Beginner-Intermediate)

Using the California housing dataset, train Ridge with 10 different alpha values: [0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000]. For each, record training R^2 , test R^2 , and the maximum absolute coefficient. Plot three line charts (one for each metric) against $\log(\alpha)$. Identify the alpha that gives the best test R^2 and write a comment explaining where overfitting and underfitting begin.

Exercise 3 — Lasso Feature Selection (Intermediate)

Using the diabetes dataset (from `sklearn.datasets import load_diabetes`), train `LassoCV` with 5-fold CV. After fitting, print: the best alpha found, a table of all 10 features with their Lasso coefficient (sorted by absolute value), how many features were set to zero, and which feature is most important according to Lasso. Verify by checking if the top feature also has the highest correlation with the target.

Exercise 4 — Scaling Impact (Intermediate)

Using the housing dataset, train Ridge (`alpha=10`) twice: once WITHOUT `StandardScaler` and once WITH `StandardScaler`. Print both coefficient tables side by side. Then compare test RMSE for both. Write a comment explaining: which version is more reliable for feature importance interpretation and why.

Exercise 5 — Full Comparison Report (Intermediate)

Build a complete comparison report on a dataset of your choice (California housing, diabetes, or your own). Train and evaluate: Linear Regression, Ridge (CV-tuned), Lasso (CV-tuned), ElasticNet (CV-tuned). For each model report: training R^2 , test R^2 , test RMSE, number of nonzero coefficients, and best hyperparameters. Present results as a formatted table. Write a paragraph recommendation: which model would you deploy and why?

Stretch Challenge — Tier A students

Implement Ridge regression from scratch using gradient descent. Add the L2 penalty term to the gradient calculation: $dW = dW_{\text{mse}} + 2 \cdot \alpha \cdot W$ (without the intercept). Train it on the housing dataset with $\alpha=1.0$. Compare your from-scratch coefficients and RMSE against sklearn's `Ridge(alpha=1.0)`. They should match within 3 decimal places. Then implement the regularisation path from scratch: loop through 50 alpha values, fit your model, store coefficients, and plot them. Compare your path to sklearn's Ridge path.